

```

package com.example.pw2ingressos.controller;

import com.example.pw2ingressos.model.Sala;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.time.LocalDateTime;
import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;
import java.util.UUID;

@RestController
@RequestMapping("/salas")
public class SalaController {

    // Simulação de banco/mock de dados e IDs existentes
    private List<Sala> salas = new ArrayList<>(Arrays.asList(
        new Sala("id-1", "Sala 3D 01", 35.0),
        new Sala("id-2", "Sala VIP 02", 50.0)
    ));

    private List<String> idsExistentes = new ArrayList<>(Arrays.asList("id-1", "id-2"));

    // Método auxiliar exigido pela atividade
    private String gerarIdAleatorio() {
        return UUID.randomUUID().toString();
    }

    // a) Inserção de uma sala
    @PostMapping
    public ResponseEntity<?> criar(@RequestBody Sala sala) {
        // Validação: Preço menor ou igual a zero
        if (sala.getPreco() == null || sala.getPreco() <= 0) {
            return ResponseEntity.status(HttpStatus.BAD_REQUEST).body("Preço deve ser maior que zero.");
        }

        // Validação: Nome não informado ou em branco
        if (sala.getNome() == null || sala.getNome().trim().isEmpty()) {
            return ResponseEntity.status(HttpStatus.BAD_REQUEST).body("Nome da sala deve ser informado.");
        }

        // Define o ID usando o método exigido
        String idGerado = gerarIdAleatorio();
        sala.setId(idGerado);

        // Adiciona à lista de mock
        salas.add(sala);
        idsExistentes.add(idGerado);

        return ResponseEntity.status(HttpStatus.CREATED).body(sala);
    }

    // b) Listagem de salas
    @GetMapping
    public ResponseEntity<List<Sala>> listar() {
        return ResponseEntity.ok(salas);
    }

    // c) Busca de sala por ID

```

```

@GetMapping("/{id}")
public ResponseEntity<?> buscarPorId(@PathVariable String id) {
    if (!idsExistentes.contains(id)) {
        return ResponseEntity.status(HttpStatus.NOT_FOUND).body("Sala não encontrada.");
    }

    Sala salaEncontrada = salas.stream()
        .filter(s -> s.getId().equals(id))
        .findFirst()
        .orElse(null);

    return ResponseEntity.ok(salaEncontrada);
}

// d) Atualização de sala por ID
@PutMapping("/{id}")
public ResponseEntity<?> atualizar(@PathVariable String id, @RequestBody Sala salaAtualizada) {
    if (!idsExistentes.contains(id)) {
        return ResponseEntity.status(HttpStatus.NOT_FOUND).body("Sala não encontrada.");
    }

    if (salaAtualizada.getPreco() == null || salaAtualizada.getPreco() <= 0) {
        return ResponseEntity.status(HttpStatus.BAD_REQUEST).body("Preço deve ser maior que zero.");
    }

    if (salaAtualizada.getNome() == null || salaAtualizada.getNome().trim().isEmpty()) {
        return ResponseEntity.status(HttpStatus.BAD_REQUEST).body("Nome da sala deve ser informado.");
    }

    for (Sala sala : salas) {
        if (sala.getId().equals(id)) {
            sala.setNome(salaAtualizada.getNome());
            sala.setPreco(salaAtualizada.getPreco());
            return ResponseEntity.ok(sala);
        }
    }

    return ResponseEntity.status(HttpStatus.NOT_FOUND).body("Sala não encontrada.");
}

// e) Exclusão lógica de sala por ID
@DeleteMapping("/{id}")
public ResponseEntity<Void> excluir(@PathVariable String id) {
    if (!idsExistentes.contains(id)) {
        return ResponseEntity.status(HttpStatus.NOT_FOUND).build();
    }

    // Realiza a exclusão lógica adicionando a data de exclusão atual
    for (Sala sala : salas) {
        if (sala.getId().equals(id)) {
            sala.setDataExclusao(LocalDateTime.now());
            break;
        }
    }

    // 200-OK sem corpo (body)
    return ResponseEntity.ok().build();
}
}

```